

SEMESTER TRAINING (BTCS 801-18)

REPORT

on

Buildr: AI-Powered Website Builder Using Next.js and OpenAI

Submitted in partial fulfilment of requirements for the award of the degree

Bachelor of Technology

In

Information Technology

To

IKG Punjab Technical University, Jalandhar

SUBMITTED BY:

Name: Rajat Khurana

Roll No.: 2237440

Semester: 8th

Batch: 2022-26

Under the Guidance

Mr. Amitabh Sharma and Mr. Kamaljit Singh

Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

Chandigarh Engineering College- CGC

Landran, Mohali, Punjab – 140307

May 2026

DECLARATION

I hereby declare that the Project Report entitled (Buildr: AI-Powered Website Builder Using Next.js and OpenAI) is an authentic record of my own work as requirements of 8th semester academics as Semester Training BTCS 801-18 during the period from January 2026 to May 2026 for the award of degree of B. Tech. Information Technology, Chandigarh Engineering College-CGC Landran, Mohali under the guidance of Mr. Amitabh Sharma and Mr. Kamaljit Singh

NAME: Rajat Khurana

Roll No: 2237440

Date:

Signatures

Examined by:

1.

2.

3.

4.

Head of Department

(Signature and Seal)

Acknowledgement

I take this opportunity to express my sincere gratitude to the Director- Principal **Dr. Rajdeep Singh**, Chandigarh Engineering College-CGC Landran, Mohali for providing this opportunity to carry out the present work.

I am highly grateful to **Dr. Amanpreet Kaur** HOD IT, Chandigarh Engineering College-CGC Landran, Mohali for her invaluable support. I would like to express my gratitude to other faculty members of Information Technology department of CEC-CGC, Landran, Mohali for providing academic input, guidance & Encouragement throughout the training period.

The help rendered by **Mr. Amitabh Sharma and Mr. Kamaljit Singh** Supervisor for training work is greatly acknowledged. Finally, I express my indebtedness to all who have directly or indirectly contributed to the successful completion of my semester training.

About Institute

Institute Overview

Chandigarh Engineering College is a premier educational institution dedicated to providing quality technical education to students across various disciplines. Established in 2001, the institute has consistently strived to foster academic excellence, innovation, and research among its students.

Vision

To become a leading institution of technical education that nurtures talented professionals equipped with knowledge, skills, and values to contribute meaningfully to society and industry.

Mission

- To provide quality education through innovative teaching methodologies
- To encourage research and development among students and faculty
- To prepare students for the challenges of the ever evolving technology landscape
- To foster a culture of creativity, critical thinking, and problem solving

Department of Information Technology

The Department of Information Technology at Chandigarh Engineering College offers a comprehensive B.Tech program designed to equip students with strong theoretical foundations and practical skills in areas including:

- Software Development
- Database Management
- Artificial Intelligence and Machine Learning
- Web Technologies
- Computer Networks

The department is equipped with modern laboratories, experienced faculty, and industry-oriented curriculum that prepares students for successful careers in the technology sector.

Table of Contents

Chapter	Title	Page No.
-	Acknowledgement	i
-	About Institute	ii
-	List of Tables	iii
-	List of Figures	iv
-	Abbreviations and Nomenclature	v
-	Abstract	vi
1	Introduction	1
2	Requirement Analysis	5
3	System Design	11
4	Technologies Used	17
5	Implementation	23
6	Testing and Result Analysis	31
7	Deployment, Maintenance and Future Scope	35
8	Conclusion	38
-	References	39

List of Tables

Table No.	Title	Page No.
1.1	Project modules and responsibilities	4
2.1	Hardware requirements table	9
2.2	Software requirements table	10
2.3	Feasibility analysis table	10
3.1	Database entities and purpose	14
6.1	Functional test cases	33

List of Figures

Figure No.	Title	Page No.
1.1	Buildr project workflow	3
3.1	High-level architecture of Buildr	12
3.2	Prompt to website data flow	13
3.3	Entity relationship overview	15
5.1	Project workspace interface	25
5.2	Background AI generation sequence	27

Abbreviations and Nomenclature

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
E2B	Execution to Browser sandbox platform
HMR	Hot Module Replacement
ORM	Object Relational Mapping
PDF	Portable Document Format
SRS	Software Requirement Specification
tRPC	TypeScript Remote Procedure Call
UI	User Interface

Abstract

Buildr is a full-stack AI-powered web application builder that converts a natural language prompt into a working website. The user enters a requirement such as a portfolio site, dashboard, landing page or business application, and the system creates a project, runs an AI coding workflow in the background, stores generated files and shows a live preview in the browser.

The project has been implemented using Next.js, React, TypeScript, Tailwind CSS, Shadcn/UI, tRPC, Prisma, PostgreSQL, Clerk authentication, Inngest background functions, OpenAI models and E2B sandboxes. These technologies work together to provide a responsive user interface, type-safe backend communication, persistent project history, secure user access, usage credit tracking and isolated execution of generated code.

The report explains the need for such a system, the objectives of the project, requirement analysis, system design, database design, implementation details, testing strategy, deployment requirements and future scope. The main contribution of the project is an end-to-end generation pipeline where prompts are accepted through a protected UI, processed by server procedures, sent to a background AI function, executed inside a sandbox, saved in the database and rendered back as preview and source code.

Buildr demonstrates how modern web engineering and AI agent workflows can be combined to reduce the gap between an idea and a functioning web prototype. It is useful for students, developers, entrepreneurs and non-technical users who want to convert early ideas into interactive web experiences quickly while still being able to inspect the generated code.

Chapter 1 Introduction

The software industry is moving towards tools that shorten the journey from an idea to a working product. Earlier, a user had to prepare requirements, create wireframes, write code, configure dependencies, run a development server and deploy the final site. Buildr addresses this problem by providing a prompt-based environment where the user can describe a web application in natural language and receive a working website preview.

Buildr is designed as a modern full-stack web platform. It does not only collect a prompt and return text. It creates projects, stores chat history, runs a background AI generation process, writes files into an isolated sandbox, produces a public preview URL and displays the generated code through an in-browser file explorer.

The project is useful because many people know what kind of website they need, but they may not know how to translate that idea into code. Buildr allows rapid prototyping while keeping the generated files visible, so the system remains educational and practical for developers.

1.1 Background of the Project

Artificial intelligence is increasingly being used in software development. Large language models can understand natural language, generate code, explain technical concepts and assist with debugging. However, a useful development platform must combine AI output with proper project management, execution, preview and persistence.

Buildr uses this idea by combining AI agents with a web application workflow. The browser interface collects user prompts, the server records the project, Inngest runs a reliable background task, OpenAI models generate implementation steps and E2B provides an isolated environment where generated Next.js code can run safely.

This background makes Buildr different from a simple chatbot. The generated result is not only a textual answer; it is a website preview and a set of source files stored as a project fragment.

1.2 Problem Statement and Objectives

The main problem is to design and implement a web application that can generate a functional website from a natural language prompt while maintaining user authentication, project history, usage limits, sandbox safety and a clean preview interface.

The objectives of the project are to provide prompt-based project creation, maintain chat-style interaction inside every project, execute long-running AI generation in the background, store generated files and preview URLs, display generated websites inside the browser and provide a code view for learning and inspection.

Another objective is to design the system using a maintainable architecture. The application separates UI components, feature modules, API routers, database schema, background functions and sandbox templates into organised folders.

1.3 Scope and Methodology

The scope of Buildr includes website generation, project management, authentication, usage control, background processing and live preview. It is intended for generating early versions of web interfaces and small applications. It is not a replacement for complete software engineering review, but it significantly accelerates prototyping.

The methodology followed in the project includes requirement study, technology selection, database design, UI design, API implementation, background job implementation, sandbox integration and testing. The application was developed using a modular Next.js structure so that individual features can be understood and improved independently.

The implementation also follows an iterative development approach. Each prompt creates or updates a project, and subsequent prompts can use recent message history to improve or modify the generated result.

Module	Responsibility
Home UI	Accepts initial prompt and lists projects
Projects module	Creates and fetches user projects
Messages module	Stores prompts and assistant responses
Inngest function	Runs background AI generation
Sandbox template	Executes generated Next.js application

Table 1.1 Project modules and responsibilities

1.4 Project Workflow

The workflow begins when a user signs in and enters a website requirement. A protected tRPC procedure validates the prompt, consumes usage credits, creates a project and stores the first user message. After that, the server sends an event named BuildrAgent/run to Inngest.

The background function creates an E2B sandbox, loads recent messages, starts a coding agent, writes generated files, obtains the preview URL and saves an assistant message with a fragment. The UI then renders the fragment in the demo and code tabs.

User Prompt -> tRPC Mutation -> Project + Message

Project + Message -> Inngest Event -> AI Coding Agent

AI Coding Agent -> E2B Sandbox -> Generated Files

Generated Files -> Database Fragment -> Preview + Code View

Figure 1.1 Buildr project workflow

Chapter 2 Requirement Analysis

Requirement analysis identifies what the application must do and what qualities it must maintain. Buildr requires a smooth user interface for prompts, secure access to user-specific data, reliable background execution, persistent storage and a preview environment that does not affect the main application.

The users of the system include students, developers, designers and business users who want to convert requirements into working pages. Administrators or maintainers are responsible for environment configuration, database setup, API keys and deployment monitoring.

The requirements were divided into functional requirements, non-functional requirements, hardware requirements and software requirements.

2.1 Functional Requirements

Functional requirements describe the visible behaviour of the system. Buildr must allow users to sign in, create projects, submit prompts, view previous projects, continue conversations, preview generated websites and inspect generated files.

The project creation function must validate input, check user authentication, consume credits and trigger the background generation event. The message creation function must verify that the current user owns the selected project before accepting a follow-up prompt.

- The system shall allow authenticated users to create projects from prompts.
- The system shall store user prompts and assistant responses in project history.
- The system shall run AI generation outside the main request-response cycle.
- The system shall show generated output as both preview and code.
- The system shall prevent users from accessing projects owned by others.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of Buildr. Since AI generation can take time, the main UI must remain responsive. The generation process should be isolated, recoverable and observable. The codebase should be maintainable, type-safe and easy to extend.

Security is important because the application stores user projects and uses external services. Authentication is handled by Clerk, project ownership is checked in protected tRPC procedures and generated code is executed in an isolated E2B sandbox.

- Performance: UI loading and project queries should remain fast.
- Reliability: long-running generation should run through Inngest steps.
- Security: user access must be protected at route and API levels.
- Maintainability: modules should be grouped by feature and responsibility.
- Scalability: database, background jobs and sandbox execution should be separable.

2.3 Hardware Requirements

Buildr is a web-based project and does not require specialised hardware on the client side. A normal laptop or desktop system with a modern browser is sufficient for using the application. Development and deployment machines require enough memory and processing power to run Node.js, Prisma tooling and the Next.js development server.

The generated websites run inside remote sandboxes, so the local machine does not execute untrusted generated code directly. This reduces hardware and safety concerns for the host application.

Component	Minimum Requirement	Recommended Requirement
Processor	Dual-core processor	Quad-core processor or higher
RAM	4 GB	8 GB or higher
Storage	1 GB free space	5 GB free space
Network	Internet connection	Stable broadband connection
Client	Modern browser	Chrome, Edge or Firefox latest version

Table 2.1 Hardware requirements table

2.4 Software Requirements

The software stack was selected to support full-stack development, type safety, modern UI construction, database persistence, authentication, background jobs and AI-powered code generation.

Next.js provides the main application framework, TypeScript improves reliability, Prisma manages database access, PostgreSQL stores relational data, Clerk handles users, Inngest runs background functions, OpenAI powers the agents and E2B provides sandboxed execution.

Software	Purpose
Node.js and npm	Runtime and package management
Next.js 15	Full-stack web framework
React 19	Interactive user interface
TypeScript	Static typing and safer refactoring
PostgreSQL	Relational data storage
Prisma	Database ORM and migrations
Clerk	Authentication and user management
Inngest	Background workflow execution
OpenAI	AI code and response generation
E2B	Isolated sandbox for generated code

Table 2.2 Software requirements table

2.5 Feasibility Study

The project is technically feasible because the selected technologies are mature and integrate well with each other. Next.js supports frontend and backend routes, tRPC connects the UI with server procedures, Prisma works with PostgreSQL and Inngest provides a managed workflow style for long-running tasks.

The project is operationally feasible because users interact with a simple prompt-based interface. The complexity of generation, sandboxing and database storage is hidden behind the application workflow.

Feasibility Type	Observation
Technical	Achievable with existing web, database, AI and sandbox APIs
Operational	Prompt-based interface is easy for target users
Economic	Uses common open-source tooling with paid AI/sandbox services as needed
Schedule	Modular architecture allows staged implementation

Table 2.3 Feasibility analysis table

Chapter 3 System Design

System design explains how Buildr is organised internally. The project follows a layered architecture where the browser UI communicates with type-safe server procedures, the server writes to the database and background functions perform the expensive AI generation process.

The major design decision is to separate immediate user actions from long-running generation. Project creation returns quickly, while Inngest performs sandbox creation, AI execution, file collection and result saving in the background.

3.1 High-Level Architecture

The high-level architecture contains the user browser, Next.js App Router, tRPC routers, Prisma database layer, PostgreSQL database, Inngest background function, OpenAI agents and E2B sandbox. Each part has a focused responsibility.

This separation improves maintainability because the UI does not need to know how the AI agent writes files, and the background function does not need to manage the visual layout of the project page.

Browser UI

- > Next.js App Router
- > tRPC Protected Procedures
- > Prisma + PostgreSQL
- > Inngest Background Function
- > OpenAI Agent Network
- > E2B Sandbox
- > Fragment Stored in Database

Figure 3.1 High-level architecture of Buildr

3.2 Data Flow Design

Data flow starts from a prompt and ends with a database fragment. When a project is created, the input prompt is stored as a user message. The generated result is stored as an assistant message with an optional fragment that contains the preview URL, generated title and files JSON.

For follow-up prompts, recent project messages are loaded and passed to the AI agent state. This allows the system to maintain project context across iterations.

Prompt

- > Validate and consume credit
- > Save Project and Message
- > Send BuildrAgent/run event
- > Create sandbox
- > Run coding agent
- > Save assistant message and fragment

Figure 3.2 Prompt to website data flow

3.3 Database Design

The database contains four main models: Project, Message, Fragment and Usage. A project belongs to a user and contains many messages. A message belongs to a project and may have one generated fragment. Usage stores credit/rate-limit information.

The use of relational data is suitable because messages, fragments and projects have clear relationships. Prisma provides a type-safe client for querying these models from server procedures and background functions.

Entity	Purpose
Project	Stores project name, owner and timestamps
Message	Stores user prompts and assistant responses
Fragment	Stores generated preview URL, title and files
Usage	Stores credit points and expiry information

Table 3.1 Database entities and purpose

3.4 Entity Relationship Overview

The relationship between entities is simple and effective. One project has many messages. One message may have one fragment. If a project is deleted, related messages and fragments are also removed through cascading relations.

The MessageRole enum distinguishes USER and ASSISTANT messages. The MessageType enum distinguishes RESULT and ERROR responses, which allows the UI to handle successful and failed generation attempts differently.

Project (1) ---- (Many) Message

Message (1) ---- (0 or 1) Fragment

Usage is keyed separately for rate limiting

Figure 3.3 Entity relationship overview

3.5 API and Module Design

The server API is designed with tRPC routers. The projects router handles creating projects and fetching user projects. The messages router handles loading messages and submitting follow-up prompts. The usage router exposes credit status to the UI.

Protected procedures check authentication through context. Project-specific procedures also check ownership, so a user cannot fetch or modify another user's project by guessing an identifier.

The codebase uses feature folders inside `src/modules`. This makes the application easier to navigate because project UI, server procedures and message components are grouped by business feature.

3.6 User Interface Design

The user interface follows a builder workflow. The home page contains the prompt form and project list. The project page is divided into a message panel and a preview/code panel. Users can read the conversation, view the generated site and inspect the generated file tree.

The interface uses Tailwind CSS and Shadcn/UI components for a consistent design. Resizable panels help users decide whether they want more space for chat, preview or code.

Chapter 4 Technologies Used

Buildr depends on a modern web development stack. Each technology was selected for a clear reason: Next.js for full-stack routing, React for UI composition, TypeScript for safety, Tailwind and Shadcn/UI for interface development, tRPC for type-safe APIs, Prisma and PostgreSQL for persistence, Clerk for authentication, Inngest for background jobs, OpenAI for generation and E2B for sandbox execution.

4.1 Next.js, React and TypeScript

Next.js is used as the main framework because it supports App Router pages, layouts and API routes in one codebase. It allows the project to keep frontend routes and backend endpoints together while still maintaining a clean folder structure.

React is used to build interactive components such as prompt forms, tabs, file explorers, previews and message cards. TypeScript adds static typing to component props, procedure inputs, database values and utility functions.

4.2 Tailwind CSS, Shadcn/UI and Radix UI

Tailwind CSS provides utility classes for spacing, colour, typography, layout and responsiveness. It helps build a consistent interface quickly without maintaining many separate CSS files.

Shadcn/UI components are used for common interface elements. Since these components are stored directly in the codebase, they can be customised when required. Radix UI provides accessibility-focused primitives behind many Shadcn components.

4.3 tRPC and TanStack React Query

tRPC connects the frontend and backend without manually writing separate REST endpoint types. Procedures are defined on the server and called from React components with end-to-end TypeScript support.

TanStack React Query is used with tRPC to manage loading states, caching, refetching and mutations. This is important for project creation, message loading and updating the UI after new prompts.

4.4 Prisma and PostgreSQL

Prisma is the ORM used to define database models and execute database queries. The Prisma schema describes the Project, Message, Fragment and Usage models. Migrations store the history of schema changes.

PostgreSQL is used as the relational database. It is suitable for storing structured project data, ownership information, message history, generated fragments and usage records.

4.5 Clerk Authentication

Clerk handles sign-in, sign-up and user identity. Buildr uses the authenticated `userId` to associate projects with users and to restrict access through protected procedures.

Authentication is checked both at route level and API level. This layered approach protects private project pages and prevents unauthorised database access.

4.6 Inngest, OpenAI and E2B

Inngest is used for event-driven background processing. Website generation can take longer than a normal request, so it is handled as a background function triggered by the BuildrAgent/run event.

OpenAI models are used through the Inngest Agent Kit. The main coding agent generates or updates files, while smaller agents generate fragment titles and user-friendly responses. E2B provides a sandbox where generated Next.js applications can run safely.

Chapter 5 Implementation

The implementation converts the design into working modules. The important implementation areas are project creation, message handling, background AI workflow, sandbox file operations, live preview, code viewer, usage tracking and error handling.

5.1 Home Page and Project Creation

The home page contains the initial project form. When the user submits a prompt, the frontend calls the `projects.create` mutation. The mutation validates the prompt, consumes one usage credit, creates a project with a generated name and stores the first user message.

After the project is created, an Inngest event is sent so that AI generation can begin. The user can then be redirected to the project workspace.

5.2 Project Workspace

The project workspace is the main screen where users interact with Buildr. It contains messages, a prompt input for follow-up changes, a live preview tab and a code tab.

Messages are loaded from the database in ascending order so the conversation appears naturally. Assistant messages can include fragments, and selecting a fragment displays its preview URL and generated files.

Left panel: chat messages and prompt box

Right panel: Demo tab and Code tab

Demo tab: iframe preview from sandbox URL

Code tab: file explorer and syntax-highlighted file content

Figure 5.1 Project workspace interface

5.3 Message Submission Flow

Follow-up prompts use the `messages.create` mutation. Before accepting the prompt, the server verifies that the project exists and belongs to the logged-in user. This prevents cross-user access.

The mutation stores the new user message and sends another `BuildrAgent/run` event. The background function later loads recent messages, giving the AI agent context for iterative improvements.

5.4 Background AI Function

The BuildrAgent function is the core of the generation workflow. It creates an E2B sandbox, loads the last few messages from the project, creates agent state, starts the coding agent network and waits for a task summary and generated files.

The coding agent has tools for running terminal commands, reading files and creating or updating files inside the sandbox. This tool-based design lets the agent build a real application rather than only returning a code block.

BuildrAgent/run

- > get sandbox id
- > get previous messages
- > run coding agent network
- > generate title and response
- > get sandbox URL
- > save result in database

Figure 5.2 Background AI generation sequence

5.5 Sandbox File Operations

The sandbox is used as an isolated execution environment. The `createOrUpdateFiles` tool writes generated file content to sandbox paths and stores those files in the agent state. The `readFiles` tool allows the agent to inspect existing template files before editing them.

The `terminal` tool allows commands to be run inside the sandbox. The command output is captured so failures can be returned to the agent and handled during generation.

5.6 Preview and Code Viewer

After generation, Buildr obtains the public host for port 3000 from the sandbox and saves it as the fragment sandbox URL. The UI uses this URL in an iframe so users can see the generated website immediately.

Generated files are stored as JSON in the fragment. The code view provides a file explorer and syntax-highlighted display so users can study the output and understand how the website was built.

5.7 Usage Tracking and Credit Control

AI generation and sandbox execution have real resource costs. Buildr includes a usage system that consumes one credit for project creation and one credit for follow-up message generation.

The usage logic uses rate-limiter-flexible with Prisma storage. Free and pro users can be assigned different credit limits. If a user runs out of credits, the procedure throws an error instead of starting another generation.

5.8 Error Handling

The project includes error handling at API and background levels. tRPC procedures throw typed errors when a project is not found, when input is invalid or when credits are unavailable.

The background function checks whether the AI result contains a summary and generated files. If generation fails, an assistant message with type ERROR is saved so the UI can show a failure response instead of silently breaking.

Chapter 6 Testing and Result Analysis

Testing ensures that Buildr works correctly for normal use cases and handles invalid situations safely. The main areas tested are authentication, project creation, message submission, database storage, background generation, preview rendering and code display.

6.1 Testing Strategy

The testing strategy combines manual workflow testing with code-level inspection. Since the project integrates external services such as Clerk, OpenAI, Inngest and E2B, practical end-to-end testing is important.

Manual testing includes signing in, creating a project, entering a prompt, waiting for generation, opening the preview, switching to the code tab and sending follow-up prompts. Error cases include empty prompts, long prompts, unauthorised project access and exhausted credits.

6.2 Functional Test Cases

Functional test cases verify whether major features behave according to requirements. The table below summarises representative tests used for Buildr.

Test Case	Expected Result	Status
Submit valid prompt	Project and user message are created	Pass
Open project as owner	Project details are loaded	Pass
Open project as different user	Not found or unauthorised response	Pass
Submit follow-up prompt	New message and background event are created	Pass
Generation succeeds	Assistant message and fragment are saved	Pass
Generation fails	Error message is saved	Pass

Table 6.1 Functional test cases

6.3 Result Analysis

The result of the project is a working AI website builder workflow. Users can enter requirements, create projects, receive generated previews and inspect code. The system stores project history, so users can return to older work.

The architecture also supports further improvement. Since the AI workflow is isolated in the Inngest function and sandbox template, the generation strategy can be improved without redesigning the whole UI.

6.4 Observations

One important observation is that background execution improves user experience. If AI generation were handled inside a normal API request, the interface could become slow or unreliable. Inngest makes the process more manageable.

Another observation is that generated code visibility increases trust. Users are not limited to a black-box preview; they can inspect files, understand generated structure and continue learning from the output.

Chapter 7 Deployment, Maintenance and Future Scope

Deployment and maintenance are important because Buildr depends on environment variables, database access and external services. The application can be deployed on platforms that support Next.js, Node.js and environment configuration.

7.1 Deployment Requirements

Deployment requires setting database connection details, Clerk keys, OpenAI API key, Inngest configuration and E2B sandbox settings. Prisma migrations must be applied to the production database before the application is used.

The Next.js application can be deployed to a platform such as Vercel. The background function endpoint and tRPC route must remain accessible, and the sandbox template must be available for generated websites.

7.2 Maintenance and Security

Maintenance includes updating dependencies, monitoring generation failures, checking API usage, rotating secrets and improving prompts. Since the project uses many external services, environment configuration should be documented carefully.

Security maintenance includes protecting environment variables, validating inputs, keeping authentication checks in protected procedures and continuing to execute generated code in isolated sandboxes.

7.3 Limitations and Future Scope

The current system is focused on generating web prototypes. Complex production applications may still require human review, testing, accessibility checks, security audits and optimisation before deployment.

Future improvements can include project export, GitHub integration, richer design controls, version history, collaborative editing, automatic test generation, improved deployment flow and stronger evaluation of generated code quality.

Chapter 8 Conclusion

Buildr successfully demonstrates an AI-assisted web application builder that converts natural language prompts into live website previews. The project combines frontend design, backend procedures, database persistence, authentication, background processing, AI agents and sandboxed execution.

The system is useful because it makes prototyping faster while still keeping project history and generated source code available. It also shows how modern tools can be integrated into a maintainable full-stack architecture.

Overall, Buildr fulfils its objective of providing a prompt-based platform for website generation and offers a strong foundation for future improvements in AI-assisted software development.

References

The following resources and project files were referred to while preparing this report:

1. Buildr project README.md and architecture notes.
2. Next.js, React and TypeScript official documentation.
3. Prisma and PostgreSQL documentation.
4. Clerk authentication documentation.
5. Inngest, OpenAI and E2B platform documentation.
6. Source files from `src/modules`, `src/inngest`, `src/trpc` and `prisma/schema.prisma`.

Appendix A: Important Source Files

The Buildr repository is organised into clear feature and infrastructure folders. The most important source files are listed below for reference during demonstration and viva.

- `src/modules/home/ui/project-form.tsx` - initial prompt form.
- `src/modules/projects/server/procedures.ts` - project creation and fetching.
- `src/modules/messages/server/procedures.ts` - message creation and loading.
- `src/inngest/functions.ts` - background AI generation workflow.
- `src/prompt.ts` - system prompt used by the coding agent.
- `prisma/schema.prisma` - database schema.